

05_Computational_Learning_Theory

Monday, May 4, 2026 2:13 PM

The computational Learning theory can be seen as the techniques to study the complexity (and feasibility) of Machine learning tasks.

Unlike traditional functions, a learning algorithm, f_A , does not simply output a single value; instead, its task is to generate a new function, C , called a **classifier** (or model).

Basically f_A takes a sequence of labeled data as input and produces a classifier as its output. This classifier then acts as a program that receives new data as input and is tasked with finding its appropriate label.

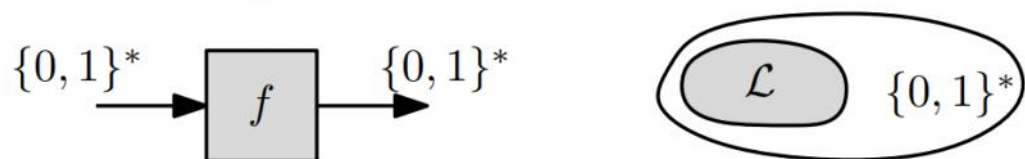
Therefore, the aim of CLT is to formalize which functions/task are easy or difficult to learn, and how many resources (both data and time) are required to do so."

PAC Learning

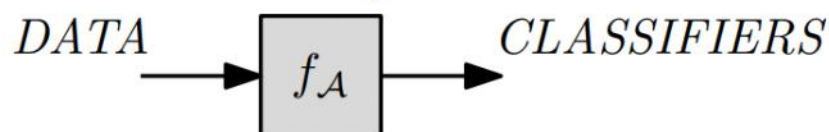
As of now we have two time of languages: a function from a string to a string.
Or a Binary-Classification problem: a language

Learning Problems as Computational Problems

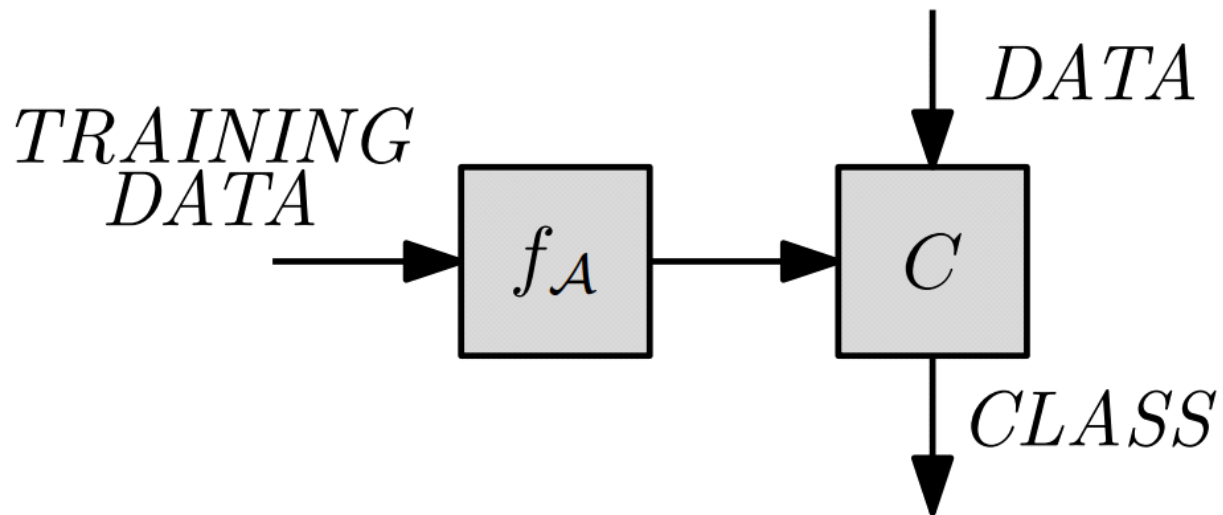
- We have so far taken functions and languages as our notions of **computational tasks**:



- Is it that **learning problems**, among the most crucial tasks in AI, can be seen as computational problems?
- The answer is positive: any learning algorithm $\mathcal{A}$ actually computes a function f_A whose input is a finite sequence of *labelled data* and whose output can be seen as a *classifier*:



Instead of a string input and a string output. We have Data as input and a Classifier as output. Where the classifier is a function it self



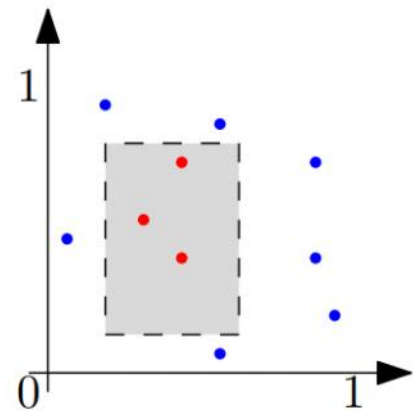
C is the classifier produced by the function f_A

Training data are labelled

While the input data to C , are not labeled, as it is the purpose of the classifier to label them

An Example Problem

Suppose that the data the algorithm $\mathcal{A}$ takes in input are points $(x, y) \in \mathbb{R}_{[0,1]} \times \mathbb{R}_{[0,1]}$ (where $\mathbb{R}_{[0,1]}$ is the set of real numbers between 0 and 1). These are labelled as positive or negative depending on they being inside a rectangle



Algorithm $\mathcal{A}$ does not receive the target rectangle as an input; instead, it is provided only with a **finite sequence of labeled data points**. These points are etichettate as **0 or 1** based on whether they are located **inside or outside** the boundaries of the unknown target rectangle.

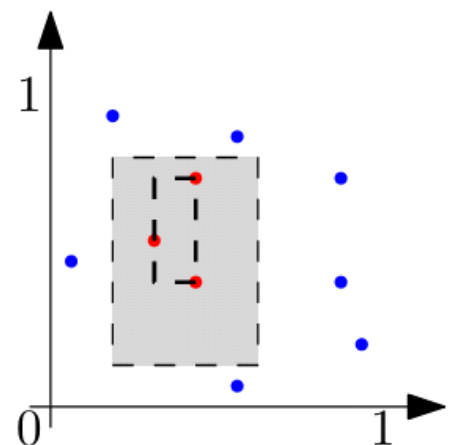
The goal of the algorithm is then to **guess a classifier**—namely, a new rectangle—that fits the received data points with high accuracy, despite never having seen the original rectangle itself.

- ▶ The algorithm $\mathcal{A}$ should be able to guess a classifier, namely a *rectangle*, based on the labelled data it received in input.
- ▶ It knows that the data are labelled according to a rectangle, R but it does not know *which rectangle* is being used.
- ▶ The algorithm $\mathcal{A}$ cannot guess the rectangle R with perfect accuracy if the data it receives in input are too few. As the data in D grow in number, we would expect the rectangle $f_{\mathcal{A}}(D)$ to converge to R , wouldn't we?
- ▶ $\mathcal{A}$ *does not know* the distribution $\mathbf{D}$ from which the points (x, y) are drawn.
 - ▶ It is supposed to “do the job” for each possible distribution $\mathbf{D}$.

This specific example of a machine learning algorithm $\mathcal{A}$ that generates a Rectangle Classifier. It's called $\mathcal{A}_{\text{BFP}}$. Notice that this is not an universal recognized name to referring to this problem

The Algorithm $\mathcal{A}_{\text{BFP}}$

- ▶ We could define an Algorithm $\mathcal{A}_{\text{BFP}}$ as follows:
 1. Given the data $((x_1, y_1), p_1), \dots, ((x_n, y_n), p_n)$;
 2. Determine the smallest rectangle R including all the positive instances;
 3. Return R .
- ▶ In the problem instance from the previous slides, one would get, when running $\mathcal{A}_{\text{BFP}}$, the little bold rectangle. Of course, the result is always a sub-rectangle of the target rectangle.



The **ABFP algorithm** always generates a **sub-rectangle** of the target T because it is specifically defined to output the **smallest rectangle R** that encloses all positive training instances. Since every positive point is, by definition, located within the boundaries of the target rectangle T , the tightest possible enclosure of those points cannot extend beyond T .

As a result, the classifier R never commits a "False Positive" error; instead, all errors are **false negatives** caused by points that fall in the gap between the edges of the learned rectangle R and the actual boundaries of the target T .

The **probability of error** for a hypothesis (rectangle R) relative to a target (rectangle T), and it highlights a fundamental principle of **PAC learning**: we prioritize the accuracy of predictions over the geometric "closeness" of the model.

Here is a detailed breakdown of the components:

The Error Formula: $error_{D,T}(R) = Pr_{x \sim D}[x \in (R - T) \cup (T - R)]$

This formula calculates the **probability** that a new point x , drawn from the environment's distribution D , will be misclassified.

- **$(R - T)$ (False Positives):** These are points that fall inside your guessed rectangle R but are actually outside the target T . The algorithm wrongly labels them as "positive."
- **$(T - R)$ (False Negatives):** These are points that are inside the target T but were missed by your rectangle R . The algorithm wrongly labels them as "negative."
- **The Union $(R - T) \cup (T - R)$:** This represents the **entire region of disagreement** (the symmetric difference) between your guess and the truth.
- **$Pr_{x \sim D}$:** This is the most critical part. It means the error is not measured by the physical area of the disagreement, but by the **likelihood** that the distribution D will actually place a point in that area.

Geometric vs. Probabilistic Convergence

The previous statement explains that as you get more data, R might not "look" like T , but it will "act" like T .

- **Geometric Discrepancy:** Imagine the target rectangle T is very large, but the distribution D only generates points in the center of T . The algorithm will create a small rectangle R around those center points. Geometrically, R is much smaller than T and has not "approached" the target's actual boundaries.
- **Probabilistic Accuracy:** Even if R is geometrically smaller than T , if the distribution D **never** generates points in the outer edges of T , then the algorithm will **never** make a mistake in those areas.

In conclusion: The goal of PAC learning is not to perfectly reconstruct the hidden target T , but to find a hypothesis R that performs correctly on the data it is likely to encounter. As long as the **probability** of a point falling into the error region $(T - R)$ approaches zero, the algorithm is considered successful, even if the rectangles themselves never perfectly overlap.

Theorem

For every distribution $\mathbf{D}$, for every $0 < \epsilon < \frac{1}{2}$ and for every $0 < \delta < \frac{1}{2}$, if $m \geq \frac{4}{\epsilon} \ln\left(\frac{4}{\delta}\right)$, then

$$Pr_{D \sim \mathbf{D}^m} [error_{D,T}(\mathcal{A}_{BFP}(D)) < \epsilon] > 1 - \delta$$

This theorem provides the **sample complexity** for the axis-aligned rectangle problem, formally proving

that the A_{BFP} algorithm is a **PAC-learner**. It establishes exactly how many training examples (m) are required to ensure that the algorithm's guess is "Probably Approximately Correct". Here is a detailed breakdown of the theorem's components and the reasoning behind its formula.

The theorem states that for any unknown distribution D , if we provide the algorithm with a specific number of samples m , we can be highly confident in the resulting model.

- **Accuracy (ϵ):** We want the probability of making a mistake on a new point to be very small (less than ϵ).
- **Confidence (δ):** Because the training data is random, there is always a tiny chance we get a "bad" batch of data. We want the probability of this failure to be less than δ .
- **The Guarantee:** If $m \geq \frac{4}{\epsilon} \ln \frac{4}{\delta}$, then the probability that our error is greater than ϵ is less than δ . (remember that m is the number of samples)

Proof of the A_{BFP} theorem

I don't think during the exam we need to know this proof :)

The goal is to calculate how many instances (m) are required so that the error of the classifier generated by the A_{BFP} algorithm is less than ϵ with a success probability greater than $1 - \delta$.

Here are the detailed steps of the proof:

1. Simplification of the Error Function

The starting point is the general definition of error for a hypothesis R with respect to a target T :

$$\text{error}_{D,T}(R) = \Pr_{x \sim D}[x \in (R - T) \cup (T - R)]$$

As said before, the A_{BFP} algorithm produces the smallest rectangle consistent with the positive data. Since every positive point must lie inside T , the resulting hypothesis R will always be a sub-rectangle of T ($R \subseteq T$).

Consequently, the term $(R - T)$ (false positives) is null. The error formula thus simplifies to the probability that a point falls within the region of T not covered by R :

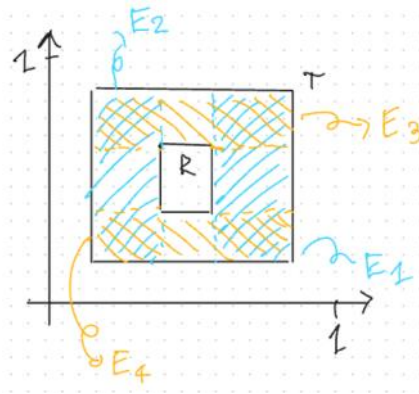
$$\text{error}_{D,T}(R) = \Pr_{x \sim D}[x \in (T - R)]$$

2. The Geometry of Error "Strips"

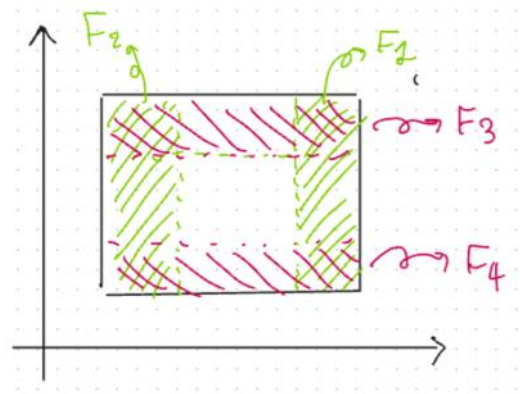
The error region $T - R$ can be visualized graphically as the union of four rectangular strips (E_1, E_2, E_3, E_4) located along the inner edges of the target rectangle T .

The idea of the proof is to define four theoretical target regions (F_1, F_2, F_3, F_4) along the sides of T such that the probability of a point falling into each of them is exactly $\epsilon/4$:

$$\Pr_{x \sim D}[x \in F_i] = \epsilon/4$$



The (E_1, E_2, E_3, E_4) error strips are linked to the rectangle R . And they are exactly the areas not "covered" by the classifier R



The (F_1, F_2, F_3, F_4) are not linked to the classifier R .
Instead we define them such that the probability of a point falling into each of them is exactly $\epsilon/4$

3. The Success Condition (The Logical Implication)

Learning success (i.e., having a total error $< \epsilon$) is guaranteed if the algorithm receives at least one positive example in each of the four regions F_i .

- If a point falls in F_i , the rectangle R (being the minimum rectangle that includes all positives) must expand to include that point.
- This forces the actual error strip E_i to become smaller than the strip F_i ($E_i \subseteq F_i$).
- If this happens for all four sides, the total error (the union of the E_i) will be less than the sum of the probabilities of the F_i :

$$\text{error}_{D,T}(R) \leq \sum_{i=1}^4 \Pr[x \in F_i] = 4 * \frac{\epsilon}{4} = \epsilon$$

4. Calculation of the Failure Probability

The algorithm fails (i.e., the error is $\geq \epsilon$) only if at least one of the four regions F_i is never "hit" by any point in the training set.

Probability that a single point does not fall into a specific region F_i : $1 - \epsilon/4$ (by the definition of F_i).

Probability that none of the m points fall into that specific region F_i : $\left(1 - \frac{\epsilon}{4}\right)^m$.

Union Bound: The probability that failure occurs in any of the four regions is at most the sum of the individual failure probabilities: $4 \times \left(1 - \frac{\epsilon}{4}\right)^m$.

5. Analytical Derivation of the Bound for m

To satisfy the PAC condition, we want this failure probability to be less than δ . Using the mathematical approximation $(1 - x)^m \leq e^{-mx}$, we proceed as follows:

$$4 \cdot e^{-\frac{m\epsilon}{4}} \leq \delta$$

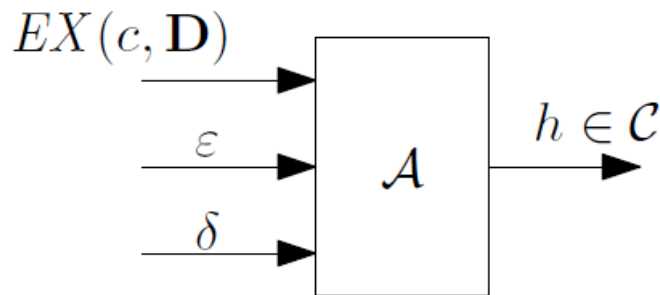
Isolating m through the algebraic steps:

- $e^{-\frac{m\epsilon}{4}} \leq \delta/4$
- Taking the logarithms: $-\frac{m\epsilon}{4} \leq \ln(\delta/4)$, which is equivalent to $\frac{m\epsilon}{4} \geq \ln(4/\delta)$
- Finally: $m \geq 4/\epsilon \ln(4/\delta)$.

This derivation proves that if we have a number of samples m that grows linearly with respect to the inverse of the precision $(1/\epsilon)$ and logarithmically with respect to the inverse of the

confidence $(1/\delta)$, the probability of not obtaining an accurate hypothesis becomes negligible. This confirms that axis-aligned rectangles are efficiently PAC-learnable.

The General Model — Terminology



Fundamental Components

- **Instance Space (X):** The set containing the encodings of all possible objects the learner intends to classify. For example, in a geometric problem, X could be the set of all points in a 2D plane (R^2).
- **Distribution (D):** A probability distribution over the instance space X through which data points are generated. It is **unknown to the learner**. It determines the likelihood of encountering specific instances; not all elements necessarily have the same probability of being drawn.
- **Concepts:** These are subsets of the instance space (X), representing specific collections of objects that share a property.
Example: a Concept in R^2 can be all the points that are in a specific rectangular area. Because there are multiple concept. we can have a different concepts for each possible shape. And for each shape different concepts for the position of that shape in the plane.
- **Concept Class (C):** A collection of concepts. These are the specific types of properties or rules that an algorithm is designed to handle or recognize because they are "sufficiently simple to describe".
Example: a concept class in R^2 can be all the rectangle you can draw in the plane
- **Target Concept ($c \in C$):** The specific, hidden rule or subset within the concept class that the learner is trying to identify.
Example: a target concept class in R^2 can be all the points that are in a specific rectangular area.
As you see Target concept and concept are the same thing. The only different is There are multiple concept but only one target concept we care.

Note: there is no difference between a concept and a classifier. Once we know the rules that define a concept, we can assess if any element of x is part of that concept or not

The Learning Process

- **Learning Algorithm (A):** A computational task (often viewed as a Turing Machine) that takes labeled data as input to generate a model. It must be designed to learn concepts from C without initially knowing the target concept c or the distribution D .
- **Oracle ($EX(c, D)$):** Rather than just a static set of data, this is a **procedure** (or "oracle") that the algorithm calls. Each call returns a random element x drawn from X according to distribution D , along with a label indicating whether x belongs to the target concept c or not.
Even if formally it is a procedure, in reality it is a file or a database, containing the labeled data that will

become our "Training Set" and "Test Set"

- **Hypothesis (h):** The output of the learning algorithm ($h \in \mathcal{C}$). It is the classifier or "approximation" that the algorithm produces, representing its best guess of the target concept c based on the training data.

Parameters and Accuracy

- **Accuracy/Error Parameter (ϵ):** This parameter defines the **maximum allowable error**. The error of a hypothesis h is the probability that a new instance drawn from distribution D will be misclassified (falling in the region where h and c disagree). We want this probability to be less than ϵ .
- **Confidence Parameter (δ):** This represents the **probability of failure**. Because training data is drawn randomly, there is a small chance the data will be non-representative. δ is the maximum probability we allow for the algorithm to fail to find an ϵ -accurate hypothesis.
 - *Note:* Both ϵ and δ are values between 0 and 1/2.

► The **error** of any $h \in \mathcal{C}$ is defined as

$$\text{error}_{\mathbf{D},c}(h) = \Pr_{x \sim \mathbf{D}}[h(x) \neq c(x)].$$

$\Pr_{x \sim D}[h(x) \neq c(x)]$: This is the probability that a randomly chosen point x (sampled according to the distribution D) will be labeled differently by your hypothesis h than by the true concept c .

PAC Learnable

- Let $\mathcal{C}$ be a concept class over the instance space X . We say that $\mathcal{C}$ is **PAC learnable** iff there is an algorithm $\mathcal{A}$ such that for every $c \in \mathcal{C}$, for every distribution $\mathbf{D}$, for every $0 < \epsilon < \frac{1}{2}$ and for every $0 < \delta < \frac{1}{2}$, is such that

$$\Pr[\text{error}_{\mathbf{D},c}(\mathcal{A}(EX(c, \mathbf{D}), \epsilon, \delta)) < \epsilon] > 1 - \delta$$

where the probability is taken over the calls to $EX(c, \mathbf{D})$

Efficiently PAC Learnable

- If the time complexity of $\mathcal{A}$ is bounded by a polynomial in $\frac{1}{\epsilon}$ and $\frac{1}{\delta}$, we say that $\mathcal{C}$ is **efficiently PAC learnable**.
- The complexity of $\mathcal{A}$ is measured taking into account the number of calls to $EX(c, \mathbf{D})$.

- ▶ If the time complexity of $\mathcal{A}$ is bounded by a polynomial in $\frac{1}{\epsilon}$ and $\frac{1}{\delta}$, we say that $\mathcal{C}$ is **efficiently PAC learnable**.
 - ▶ The complexity of $\mathcal{A}$ is measured taking into account the number of calls to $EX(c, \mathbf{D})$.

Representation Classes

The Limitation of Basic PAC Learning

In the initial definition of **efficient PAC (Probably Approximately Correct) learning**, an algorithm A is considered efficient if its time complexity is bounded by a polynomial in only two variables: $\frac{1}{\epsilon}$ (the inverse of the error parameter) and $\frac{1}{\delta}$ (the inverse of the confidence parameter).

This is **unrealistic** because we are assuming that the learning task's difficulty is independent of the **target concept c** itself. In practice, learning a very "large" or complex concept (like a deep neural network with millions of parameters) naturally requires more time and more data than learning a "small" or simple concept (like a single rectangle). If the algorithm's time is strictly limited by a polynomial that doesn't account for the concept's complexity, it cannot scale to harder problems.

The Remedy: Binary Representation

To solve this, the theory adopts a standard computer science approach: **encoding**.

- **Instance Encoding:** Elements of the instance space X (the objects we want to classify) are represented as **binary strings**.
- **Concept Encoding:** Concepts in $\mathcal{C}$ (the rules for classification) are also represented as binary strings. This allows us to define the **size(e)** of a concept e as the number of bits required to represent it.

By doing this, learning transforms from an abstract set-theory problem into a **computational problem** that a Turing Machine or an ordinary algorithm can process.

Structural Organization: X_n and $\mathcal{C}_n$

We organize the space into a hierarchy based on a parameter n , typically representing the **dimension** or **number of variables**.

- $X = \bigcup_n X_n$: The total instance space is the union of all instances of size n .
- $\mathcal{C} = \bigcup_n \mathcal{C}_n$: The total concept class is the union of all concepts that act on instances in X_n .
- **Representation Class:** This pair $(X, \mathcal{C})$ is called a representation class.

Two example of different instance spaces

- **Boolean Representation Class:**
 - X_n : The set of all boolean vectors of a fixed length n (e.g., $\{0, 1\}^n$).
 - $\mathcal{C}_n$: The set of all subsets of these vectors that can be described by **Conjunctive Normal Forms (CNFs)** using n variables.
 - For example, if $n = 4$, a concept might be a specific formula like $x_1 \wedge \neg x_2 \wedge x_4$.
- **Real-Valued/Neural Network Representation Class:**
 - X_n : The set of vectors of real numbers of length n (R^n).
 - $\mathcal{C}_n$: Subsets of that space represented by a specific architecture, such as a **neural network** with n inputs and 1 output.

- In this case, $size(c)$ would correspond to the number of weights or internal nodes in the network.

New Definition of Efficiency

Because we now have a formal way to measure the "size" of the input (n) and the "size" of the target concept ($size(c)$), the definition of **efficient PAC learning** is updated. A representation class is efficiently PAC learnable if the algorithm A runs in time **polynomial in four parameters**:

1. n : The dimension of the input space.
2. $size(c)$: The complexity/length of the target concept.
3. $\frac{1}{\epsilon}$: The desired accuracy.
4. $\frac{1}{\delta}$: The desired confidence.

Summary of the Shift

This part is about the shift from **abstract learning** (where we don't care how a concept is described) to **represented learning** (where we care about the length of the string describing the concept). This allows the theory to distinguish between classes that are easy to learn (like **conjunctions of literals**) and classes that are computationally hard to learn (like **3-term DNF formulas**, which are not efficiently learnable unless $NP = RP$, we will see this later in the file).

Now we will do some example of concept that are efficiently PAC learnable, and concept that are not.

Learning Conjunctions of Literals

The representation class CL_n consists of all possible **conjunctions of literals** over a set of n boolean variables $x_1, \dots, x_n$. A literal is either a variable x_i or its negation $\neg x_i$. For example, in a 4-variable instance space, the conjunction $x_1 \wedge \neg x_2 \wedge x_4$ defines a specific concept (a subset of $\{0, 1\}^4$) where an assignment is positive only if x_1 is true, x_2 is false, and x_4 is true.

The Banking Example: Learning an Allowance Rule

1. Instance Space and Variables (X_n): The bank assesses applicants based on n boolean characteristics. Each applicant is an assignment $s \in \{0, 1\}^n$.

- x_1 : Has a steady job (1 = Yes, 0 = No)
- x_2 : Has existing debts (1 = Yes, 0 = No)
- x_3 : Has a family (1 = Yes, 0 = No)
- ... and so on for n variables.

2. The Target Concept (c): The bank follows a secret rule that is a **conjunction of literals**. For example, their rule might be: "We give an allowance only if the applicant **has a job** (x_1) AND **does NOT have debts** ($\neg x_2$)."

- Target Concept $c = x_1 \wedge \neg x_2$.

3. Labelled Data: The learning algorithm receives pairs (s, b) :

- **Positive Examples ($b = 1$):** Applicants who were granted the allowance.
- **Negative Examples ($b = 0$):** Applicants who were denied.

4. The Learning Algorithm at Work: The algorithm starts with the most restrictive hypothesis possible, assuming the bank's rule requires *everything* to be true and false at the same time:

- **Initial Hypothesis (h):** $x_1 \wedge \neg x_1 \wedge x_2 \wedge \neg x_2 \dots \wedge x_n \wedge \neg x_n$. (This rule currently approves nobody).

The algorithm then processes **positive examples** (approved applicants) and ignores negative ones:

- Suppose the first approved applicant has a job ($x_1 = 1$), has no debts ($x_2 = 0$), and has a family ($x_3 = 1$).
- The algorithm looks at its current rule. Since $x_1 = 1$, the literal $\neg x_1$ is contradicted, so it is **deleted**. Since $x_2 = 0$, the literal x_2 is contradicted and **deleted**. Since $x_3 = 1$, $\neg x_3$ is **deleted**.
- **Updated Rule (h):** $x_1 \wedge \neg x_2 \wedge x_3 \dots$

Now that we have defined the problem and the algorithm, we want a mathematical proof that learning a conjunction of literals is **Efficiently PAC learnable**

Proof of Efficient PAC-Learnability

The representation class of conjunctions is **efficiently PAC-learnable** because the number of samples m required to achieve a desired error ϵ and confidence δ is polynomial in n , $\frac{1}{\epsilon}$, and $\frac{1}{\delta}$. The proof relies on bounding the probability that "harmful" literals survive the refinement process.

1. Defining the Error

In this algorithm, the hypothesis h is always a "superset" of literals compared to the target c (meaning h is more restrictive). This because the algorithm starts with the most restrictive hypothesis possible, in our example the bank's rule requires *every* characteristic to be true and false at the same time: $x_1 \wedge \neg x_1 \wedge x_2 \wedge \neg x_2 \dots \wedge x_n \wedge \neg x_n$. As it refines it only after every iteration

Therefore, h will never produce a false positive (it never says "1" when c says "0"). The only possible errors are **false negatives**, where $c(x) = 1$ but $h(x) = 0$. This happens if h contains at least one literal L that is not in c and that evaluates to false for an assignment x where $c(x) = \text{true}$. The error is defined as the probability of drawing an assignment x from distribution D such that $c(x) = 1$ and $h(x) = 0$:

$$\text{error}(h) = \Pr_{x \sim D} [c(x) = 1 \wedge h(x) = 0].$$

2. Harmful Literals

A literal L is considered **harmful** if it is not in the target concept c but remains in the hypothesis h , and it significantly contributes to the error.

Formally, let $p(L) = \Pr_{x \sim D} [L(x) = 0 \wedge c(x) = 1]$.

This expression measures the probability that a specific literal L will cause an **error**—specifically, a false negative.

- $p(L)$: This is the notation assigned to the probability value for a given literal L .
- $\Pr_{x \sim D}[\dots]$: This indicates the **probability** that the event inside the brackets occurs when an instance x is drawn from the instance space according to a distribution D . This distribution is unknown to the learning algorithm.
- $L(x) = 0$: This means the literal L evaluates the instance x as **false**.
- $c(x) = 1$: This means the **target concept** c (the true rule the learner is trying to find) evaluates the instance x as **positive**.

Summary of $p(L)$: It represents the probability of drawing an example that the true rule considers positive ($c(x) = 1$), but that our literal L would "reject" by evaluating it as false ($L(x) = 0$). Because the learning algorithm for conjunctions works by **elimination**—starting with all literals and removing those contradicted by positive examples—if this literal remains in the final hypothesis, it will force

the hypothesis to output "false" for instances where the true answer is "true".

A literal is defined as **harmful** if its individual contribution to the total error is too high:

$$p(L) > \frac{\epsilon}{2n}$$

- **ϵ (epsilon)**: This is the **error parameter** (or accuracy parameter). It represents the maximum total error we are willing to tolerate in the final hypothesis.
- **n** : This is the number of boolean variables in the problem.
- **$2n$** : This is the total number of possible literals, as each of the n variables x_i can appear as either x_i or its negation $\neg x_i$.
- **$\frac{\epsilon}{2n}$** : This is the "tolerance threshold" for each individual literal.

Logic of the Limit: The total error of the hypothesis is the sum of the probabilities that any of the literals remaining in the hypothesis cause a mistake. If every "bad" literal (those not in the target concept c) that survives the learning process contributes less than $\frac{\epsilon}{2n}$ to the error, then even if all $2n$ possible literals were present, the total error would not exceed ϵ because $2n \cdot (\epsilon/2n) = \epsilon$. Therefore, a literal is **harmful** if the probability of it causing a mistake is greater than this small fraction of the total allowed error. The core of the **PAC-learnability proof** is demonstrating that, with a sufficient number of training samples, the probability that any such "harmful" literal survives without being eliminated by the algorithm is extremely low.

3. Probability of Survival (Confidence)

For a harmful literal L to stay in h after m samples, it must evaluate to **true** for every one of the m positive instances (otherwise, the algorithm would have deleted it). The probability that a literal with $p(L) > \frac{\epsilon}{2n}$ survives m independent positive samples is:

$$\Pr[L \text{ survives}] = (1 - p(L))^m \leq \left(1 - \frac{\epsilon}{2n}\right)^m$$

4. The Union Bound

There are at most $2n$ total literals that could be harmful. Using the **union bound**, the probability that *at least one* harmful literal survives in h is:

$$\Pr[\exists L \in \text{Harmful s.t. } L \text{ survives}] \leq 2n \left(1 - \frac{\epsilon}{2n}\right)^m$$

At this point, the standard mathematical inequality $(1 - x) \leq e^{-x}$ is applied. This inequality is valid for all real x and is frequently used in probability theory to simplify terms involving powers. In our specific proof, we set $x = \frac{\epsilon}{2n}$. By substituting this value into the inequality, we obtain:

$$\left(1 - \frac{\epsilon}{2n}\right) \leq e^{-\epsilon/2n}$$

Next, we raise both sides of the inequality to the power of m (representing the number of samples):

$$\left(1 - \frac{\epsilon}{2n}\right)^m \leq \left(e^{-\epsilon/2n}\right)^m$$

Finally, by utilizing the algebraic property of exponents where $(a^b)^c = a^{bc}$, the term on the right-hand side is simplified:

$$\left(e^{-\frac{\epsilon}{2n}}\right)^m = e^{-m\epsilon/2n}$$

then we get:

$$2n \left(1 - \frac{\epsilon}{2n}\right)^m \leq 2ne^{-\frac{m\epsilon}{2n}}$$

To ensure this probability is less than our confidence parameter δ , we set $2ne^{-\frac{m\epsilon}{2n}} \leq \delta$ and solve for m :

$$m \geq \frac{2n}{\epsilon} \ln\left(\frac{2n}{\delta}\right)$$

5. Accuracy Bound

If the algorithm collects at least m samples, then with probability $1 - \delta$, **no harmful literals** remain in h . If no harmful literals remain, then every literal L currently in h must satisfy $p(L) \leq \frac{\epsilon}{2n}$. The total error of the hypothesis is the sum of the probabilities that any of its literals cause a false negative:

$$\text{error}(h) \leq \sum_{L \in h} p(L) \leq (2n) \cdot \frac{\epsilon}{2n} = \epsilon.$$

Thus, the algorithm is **Probably Approximately Correct (PAC)**, and since the sample complexity m and the per-sample processing time are both polynomial in n , it is **efficiently learnable**.

Intractability of Learning DNFs

The professor talk about the machine learning algorithm A , as if it is a random algorithm. Honestly this definition doesn't make sense to a mere human like me. So I try my best to Explain this part with a simple reduction between 3-term DNF and 3-coloring Graph.

The problem of the **intractability of learning a 3-term DNF formula** is a fundamental result in computational learning theory showing that even a slight increase in the complexity of a concept class can move it from being "easy" to learn to being computationally "hard".

Definition of a 3-term DNF

A **3-term DNF** (Disjunctive Normal Form) formula over n variables $(x_1, \dots, x_n)$ is a propositional formula consisting of the OR (disjunction) of exactly three terms, where each term is an AND (conjunction) of literals. It takes the form: $T_1 \vee T_2 \vee T_3$. In this structure, each T_i is a conjunction of literals (a literal is a variable or its negation, e.g., $x_1 \wedge \neg x_2 \wedge x_4$). This class is significant because it is more expressive than simple conjunctions of literals, yet it remains far from being universal—meaning it cannot represent every possible boolean function.

The Choice of NP-Complete Problem

The proof uses the **3-colourability of graphs** as its NP-complete foundation. This problem asks if the nodes of an undirected graph G can be assigned one of three colors (say, red, green, and blue) such that

no two adjacent nodes share the same color.

Terms as Colors

A 3-term DNF takes the form $T_1 \vee T_2 \vee T_3$, where each T_i is a conjunction of literals. In the reduction, we treat each of the **three terms** as one of the **three available colors**:

- **Term T_1** represents the color **Red**.
- **Term T_2** represents the color **Blue**.
- **Term T_3** represents the color **Green**.

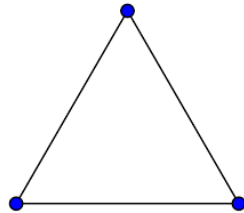
To "color" a point (a vertex) means to make it satisfy one of these three terms.

Variables as Vertices

For a graph G with n vertices, we create a representation class where the instance space consists of boolean vectors of length n . Each variable x_i in the formula corresponds directly to a **vertex v_i** in the graph.

Imagine a graph G with $n = 3$ vertices: v_1, v_2 and v_3 .

- **Vertices:** $\{v_1, v_2, v_3\}$
- **Edges:** $\{(v_1, v_2), (v_2, v_3), (v_3, v_1)\}$ (every vertex is connected to every other vertex).



The Instance Space and Variables

We define an instance space $X_3 = \{0, 1\}^3$. Each variable x_i in the boolean formula corresponds to vertex v_i .

- x_1 : Corresponds to v_1 .
- x_2 : Corresponds to v_2 .
- x_3 : Corresponds to v_3 .

Step 1: Generating the Training Data (S_+ , S_-)

The reduction function f takes our graph G and generates labeled data. We use array with the same length of the vertices to display the edges between vertices.

Example: (1,1,0) means that there is an edge between the vertex v_1 and the vertex v_2 .

We create 2 set of example

- **Positive Examples (S_+):** We create an example to ensure the formula can be "True" when no conflicts exist.
 - The all-zeros vector (0, 0, 0) is added to S_+ .
 - **Requirement:** The 3-term DNF $T_1 \vee T_2 \vee T_3$ must output **1** (True) for this input. (if there is no edge all the nodes can be colored without problem)
- **Negative Examples (S_-):** We create one negative example for every edge in the graph. These represent "active" pairs of vertices that **cannot** share the same color.
 - Edge $(v_1, v_2) \rightarrow (1, 1, 0) \in S_-$.
 - Edge $(v_2, v_3) \rightarrow (0, 1, 1) \in S_-$.
 - Edge $(v_3, v_1) \rightarrow (1, 0, 1) \in S_-$.
 - **Requirement:** The formula must output **0** (False) for these inputs. Since it is an OR of three terms, **each individual term T_i** must be False for these vectors. This is equivalent to saying that we don't want those two vertices to have the same color.

Step 2: The Learning Algorithm at Work

We assume an efficient consistent learner B . The learner's task is to find a 3-term DNF $T_1 \vee T_2 \vee T_3$ consistent with this data.

The Coloring Logic: If the graph is 3-colorable, the learner can assign each vertex to a term. Let's say:

- v_1 is **Red** (Term T_1).
- v_2 is **Green** (Term T_2).
- v_3 is **Blue** (Term T_3).

The learner constructs terms using literals to "isolate" these vertices and avoid conflicts:

- T_1 : Must be False if $x_2 = 1$ or $x_3 = 1$. Thus, $T_1 = (\neg x_2 \wedge \neg x_3)$.
- T_2 : Must be False if $x_1 = 1$ or $x_3 = 1$. Thus, $T_2 = (\neg x_1 \wedge \neg x_3)$.
- T_3 : Must be False if $x_1 = 1$ or $x_2 = 1$. Thus, $T_3 = (\neg x_1 \wedge \neg x_2)$.

Final Result: The Learned Formula

The learner outputs the hypothesis: $h = (\neg x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2)$

Verification of the Solution

To be a "consistent learner," the formula must perfectly match the data:

- **Check S_+ :** For input $(0, 0, 0)$, all negations become 1. $T_1 = 1, T_2 = 1, T_3 = 1$. The OR is **True**. (Correct).
- **Check S_- (Edge (v_1, v_2)):** For input $(1, 1, 0)$:
 - $T_1: \neg 1 \wedge \neg 0 = 0$.
 - $T_2: \neg 1 \wedge \neg 0 = 0$.
 - $T_3: \neg 1 \wedge \neg 1 = 0$.
 - The OR is **False**. (Correct). Since the formula is False, it confirms that v_1 and v_2 **do not share a term/color**.

Why This Matters

In this $n = 3$ example, the learner succeeded because a 3-coloring exists. However, if we added a fourth vertex connected to all others (a K_4 clique), no 3-term DNF could satisfy the negative edge constraints without failing on the positive example. Because finding this formula is equivalent to solving the **NP-complete** 3-colorability problem, we conclude that no polynomial-time learner can exist for this class unless $NP = RP$.

THIS IS A RESUME OF THE PROF EXPLANATION

The Foundation: RP vs. NP

The proof relies on the definition of the complexity class **RP (Randomized Polynomial time)**. A language L is in **RP** if there exists a randomized polynomial-time Turing machine that:

- If $x \in L$, the machine returns 1 with probability at least **1/2**.
- If $x \notin L$, the machine returns 0 with probability **1** (it never gives a false positive).

The goal of the proof is to show that if an efficient learner for 3-term DNFs exists, we could build such a machine to solve an **NP-complete** problem, thereby proving $NP \subseteq RP$.

The Core Theorem of Intractability

The professor establish a critical negative result: **If $NP \neq RP$, then the representation class of 3-term DNF formulas is not efficiently PAC learnable.**

While simple conjunctions of literals are efficiently PAC-learnable (meaning an algorithm can find a hypothesis that is "Probably Approximately Correct" in polynomial time), adding just two more terms to the formula makes the task intractable. This intractability specifically refers to **proper learning**, where

the algorithm is required to output a hypothesis that is itself a 3-term DNF.

Turning the Learner into a Solver

If we assume an efficient PAC learner B for 3-term DNFs exists, we can construct a randomized decision algorithm A to solve 3-colourability as follows:

1. **Call the reduction:** On input graph G , use $f(G)$ to generate the data sets (S_+, S_-) .
2. **Define the distribution:** Treat the union of $S_+ \cup S_-$ as the instance space, and assume a **uniform distribution** where every assignment in the set has the same probability of being drawn.
3. **Set "Impossible" Parameters:** Invoke the learner B with specific parameters:
 - **Confidence (δ):** Set $\delta < 1/2$.
 - **Accuracy (ϵ):** Set $\epsilon < 1/|S_+ \cup S_-|$.
4. **Force Perfection:** Because the distribution is uniform, the probability of any single point in the training set is $1/|S_+ \cup S_-|$. By setting the error parameter ϵ to be smaller than this value, we are essentially telling the learner that **it cannot make even a single mistake** on the training data. To achieve an error less than ϵ , the learner is forced to find a hypothesis that is **perfectly consistent** with every point in $S_+ \cup S_-$.

Analyzing the Outcomes

The randomized algorithm A then checks the output of the learner:

- **Case 1: G is 3-colourable.** By the reduction property, a consistent 3-term DNF exists. Since B is a valid PAC learner, it must (with probability $\geq 1 - \delta$) find a hypothesis with error $< \epsilon$. Given our choice of ϵ , this hypothesis must be perfectly consistent. Since $\delta < 1/2$, the algorithm successfully identifies the graph as 3-colourable with probability at least $1/2$.
- **Case 2: G is NOT 3-colourable.** By the reduction property, **no 3-term DNF** exists that can correctly label all points in $S_+ \cup S_-$. Consequently, the learner B can never find a consistent 3-term DNF, and the algorithm A will always return 0.

Conclusion of the Proof

Because algorithm A solves 3-colourability within the exact constraints of the class **RP**, and 3-colourability is **NP-complete**, the existence of the learner B would imply that $NP = RP$. Since this collapse of complexity classes is considered highly unlikely, we conclude that **no efficient algorithm for properly learning 3-term DNFs can exist**.

Occam's Razor in Learning

- ▶ **Occam's Razor** is the principle that, among competing explanations, the *simplest one should be preferred*.
- ▶ In machine learning, this translates into:
 - ▶ Prefer **simpler hypotheses** that fit the data;
 - ▶ Avoid unnecessarily complex classifiers.
- ▶ Intuition:
 - ▶ A hypothesis that is too complex may **overfit** the data;
 - ▶ Simpler hypotheses are more likely to **generalize**.

Consistent Learning Algorithms

- ▶ A learning algorithm $\mathcal{A}$ is **consistent** if:
 - ▶ For every training set D , it outputs a hypothesis h such that $h(x) = c(x)$ for all $(x, c(x)) \in D$.

This sentence doesn't mean that the algorithm $\mathcal{A}$ generate the same hypothesis for every training set D . If the training set is different it will be different also the hypothesis h .

Instead this sentence defines a property of the algorithm, not of the hypothesis. It means that algorithm $\mathcal{A}$ is 'good' (consistent) if, whatever dataset D you feed it, it is able to find a hypothesis h that makes no errors on those specific data.

- ▶ In other words:
 - ▶ $\mathcal{A}$ produces a hypothesis with **zero training error**.

Occam's Razor for Consistent Learners

- ▶ Suppose $\mathcal{A}$ always outputs a hypothesis consistent with D .
- ▶ Occam's Razor suggests:
 - ▶ Among all consistent hypotheses, choose the **simplest**.
- ▶ Simplicity can be formalized as:
 - ▶ **Description length** (shorter representations);
 - ▶ **Size** of the hypothesis (e.g., number of parameters).
- ▶ Key insight:
 - ▶ Fewer simple hypotheses $\Rightarrow$ less chance of fitting noise.
- ▶ This leads to:
 - ▶ Better **generalization guarantees**.

Why Simplicity Helps Generalization

- ▶ Let $\mathcal{H}$ be a finite hypothesis class.

In computational learning theory, a **hypothesis class** is the collection of all potential classifiers or "explanations" that a learning algorithm is allowed to consider when trying to solve a task.

When we say a hypothesis class is **finite**, it means that the total number of distinct hypotheses in that set is a specific, limited integer (denoted as $|\mathcal{H}|$), rather than being infinite.

- ▶ Consider a consistent hypothesis $h \in \mathcal{H}$.
- ▶ If h has high true error:
 - ▶ The probability it matches all m samples is small.
- ▶ By a union bound over $\mathcal{H}$:
 - ▶ Large hypothesis classes increase the risk of **overfitting**.
- ▶ Therefore:
 - ▶ Smaller (simpler) $\mathcal{H} \Rightarrow$ better guarantees;
 - ▶ Sample complexity depends on $\log |\mathcal{H}|$.

- ▶ Occam's principle can be formalized in the PAC framework.
- ▶ Informally:
 - ▶ If a consistent hypothesis has a **short description**, then it is probably **approximately correct**.
- ▶ This leads to the **Occam Bound**:
 - ▶ Fewer bits to describe $h \Rightarrow$ fewer samples needed.
- ▶ Consequence:
 - ▶ Learning = **compression of data**.
- ▶ Summary:
 - ▶ Consistency ensures zero training error;
 - ▶ Simplicity ensures **good generalization**.

I don't want to study this version of the theorem so for now I leave it as it is

Theorem (Occam's Razor, Cardinality Version)

Let C be a concept class and H a hypothesis class. Let L be a consistent learner for C using H . Then for all $0 < \varepsilon < 1/2$ and all $0 < \delta < 1/2$, if L is given a sample of size m drawn from $EX(c, D)$, such that

$$m \geq \frac{1}{\varepsilon} \left(\log |H_n| + \log \frac{1}{\delta} \right),$$

then L is guaranteed to output a hypothesis $h \in H_n$ that with probability at least $1 - \delta$, errs with probability at most ε . If furthermore, L is an efficient consistent learner, $\log |H_n|$ is polynomial in n and $\text{size}(c)$, and H is polynomially evaluable, then C is efficiently PAC-learnable using H .

VC Dimension

VC-Dimension (Vapnik-Chervonenkis Dimension) is a fundamental concept in computational learning theory that measures the **expressive power** or complexity of a hypothesis class H .

1. The Concept of "Shattering"

To understand VC-Dimension, one must first define what it means for a set of points to be "shattered" by a hypothesis class.

- Let H be a hypothesis class over an instance space X .
- A set of points $S = \{x_1, \dots, x_m\} \subseteq X$ is said to be **shattered** by H , if, for **every possible labeling** $\ell : S \rightarrow \{0, 1\}$, there exists a hypothesis $h \in H$ such that $h(x) = \ell(x)$ for all $x \in S$.
- In other words, if a set of m points is shattered, the hypothesis class is rich enough to realize all 2^m possible combinations of labels (positive/negative) on those specific points.

To explain the concept of **shattering**, let's use an example: the hypothesis class of **intervals on the real line**.

Imagine your hypothesis class H consists of all possible closed intervals $[a, b]$ on a line. A point is labeled **1** (positive) if it falls inside the interval and **0** (negative) if it falls outside.

Example with 2 points ($m = 2$)

Let's take two distinct points on the line, for instance, $x_1 = 1$ and $x_2 = 2$. To say that this set $S = \{1, 2\}$ is **shattered** by H , we must be able to realize all $2^2 = 4$ possible combinations of labels using only intervals:

1. **Label (0, 0):** Both negative. We can choose an interval far away, like. Both x_1 and x_2 are outside.
2. **Label (1, 0):** x_1 positive, x_2 negative. We choose the interval $[0.5, 1.5]$. Point x_1 is inside, point x_2 is outside.
3. **Label (0, 1):** x_1 negative, x_2 positive. We choose the interval $[1.5, 2.5]$. Point x_1 is outside, point x_2 is inside.
4. **Label (1, 1):** Both positive. We choose the interval $[0.5, 2.5]$. Both points are inside.

Since we successfully created every possible combination, we say the class of intervals **shatters** the set of 2 points.

Why 3 points CANNOT be shattered ($m = 3$)

For a set to be shattered, **every** possible labeling must be achievable. If we take three ordered points $x_1 < x_2 < x_3$, there are $2^3 = 8$ combinations. Consider the labeling **(1, 0, 1)**, where:

- x_1 is positive (1)
- x_2 is negative (0)
- x_3 is positive (1)

To label x_1 and x_3 as positive, our interval $[a, b]$ must necessarily contain both x_1 and x_3 . However, if an interval contains the endpoints, it must also contain everything in between, including the point x_2 . It is therefore **impossible** to have an interval that includes x_1 and x_3 but excludes x_2 . Because this specific combination cannot be realized, the class of intervals **does not shatter** any set of 3 points.

Summary

- If the hypothesis class is "rich enough" to cover every possible labeling combination for a set of points (as seen here with 2 points and intervals), that set is **shattered**.
- The **VC-Dimension** is the **maximum size** of a set that can be shattered by the class H . In this specific example, the VC-Dimension is **2**.

Definition of VC-Dimension

The **VC-Dimension** of a hypothesis class H , denoted as $VC(H)$, is defined as follows:

- It is the **maximum cardinality** of a set S that can be shattered by H .
- If it is possible to find arbitrarily large sets that can be shattered, the VC-dimension is said to be **infinite** (∞).

In practical terms, to prove that $VC(H) = d$:

1. One must demonstrate that there exists **at least one** set of size d that can be shattered by the

class.

2. One must demonstrate that **no** set of size $d + 1$ can be shattered by the class H .

In our previous example the set of all possible classifier H is $[a, b]$, and we had demonstrated that it can shatter a set of $|S| = 2$ points but not a set of $|S| = 3$

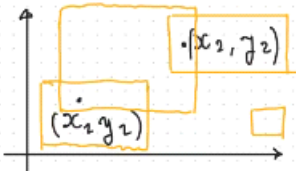
3. Practical Examples

These are several examples to illustrate how VC-Dimension reflects model complexity:

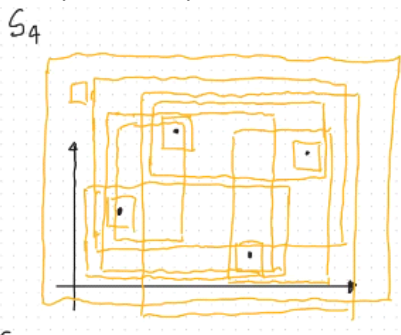
- **Intervals on $\mathbb{R}$:** that is the formal name of our previous example on intervals
- **Axis-Aligned Rectangles in $\mathbb{R}^2$:** This means drawing rectangles to create a set of these points in a cartesian plane. And we want to create all the possible sets:

Example with 2 points: we have the empty set, the set with both points and the two sets

$$S_2 = \{ \emptyset, \{(x_1, y_1), (x_2, y_2)\} \}$$



Example with 4 points



The **VC-dimension is 4**. There exists a set of 4 points that can be shattered, but no set of 5 points can be shattered by this class. Intuitively, this value corresponds to the 4 parameters (the four sides) required to define such a rectangle

- **An example of VC-dimension at ∞** is points in $\mathbb{R}^2$ divided by Polygon. It's always possible to find a polygon that correctly divides points in $\mathbb{R}^2$

POLYGONS

THESE ARE THE REGIONS OF $\mathbb{R}^2$ IDENTIFIED BY AN ARBITRARY NUMBER OF VERTICES LINKED BY EDGES, E.G.



FOR EVERY n THERE IS AN S_n WITH $|S_n| = n$ WHICH CAN BE SHATTERED



4. Why is it Important?

VC-Dimension is crucial because it provides a formal characterization of **PAC-learnability** (Probably Approximately Correct learnability):

- **Positive Result:** If a hypothesis class H has a **finite** VC-dimension d , then it is **PAC-learnable**. In this case, the sample complexity (the number of data points required to learn with high confidence and accuracy) depends linearly on d . Specifically, for any consistent hypothesis, the error will be less than ϵ with high probability if the number of samples m is proportional to

$$\frac{1}{\epsilon} \left(d \log \frac{1}{\epsilon} + \log \frac{1}{\delta} \right)$$

- **Negative Result:** If the VC-dimension is **infinite**, the class is **not PAC-learnable**. This is because a class with infinite VC-dimension can fit any "noise" or arbitrary labeling in the data, which prevents the model from generalizing to new, unseen examples.

In summary, VC-Dimension allows for the formal quantification of a model's abstraction power and determines whether a learning problem can be solved with a finite amount of data.

EXERCISES

Remember the definition of VC is the max cardinality of S that can be shattered by the Hypotesis class H

VC-DIMENSION

$$VC(H) = \max_{S \subseteq X} \{ |S| \mid S \text{ IS SHATTERED BY } H \}$$

NOTICE:

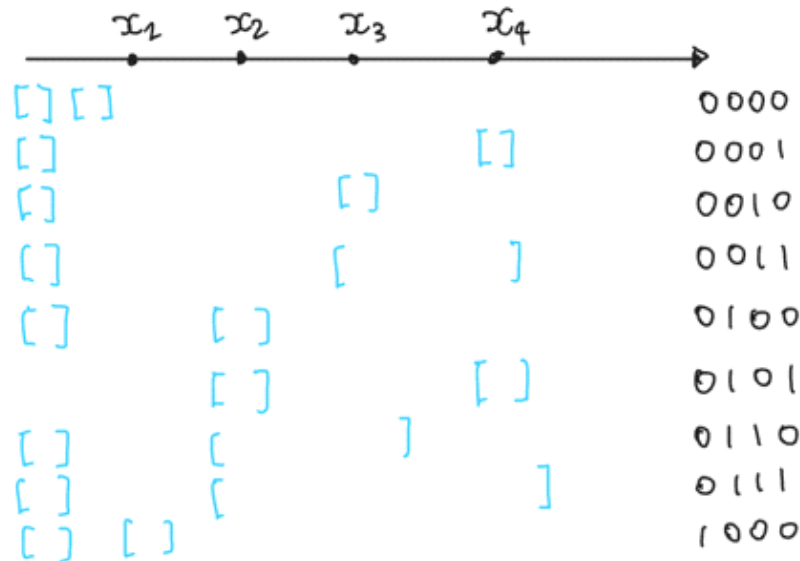
in general in these exercise the professor never select S as the worst chase Dataset set but as the best case dataset

Exercise 1

Determining the VC-Dimension of the representation class consisting of unions of two interval in $\mathbb{R}$.
We want to prove that two interval can shatter 4 numbers

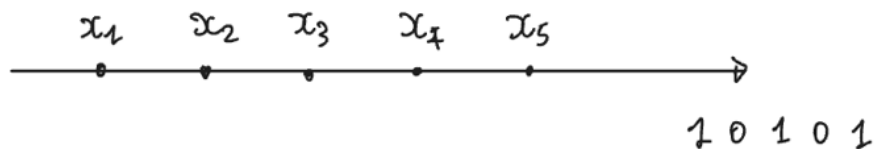
LOWER BOUND

LET US FIRST PROVE THAT $VC(\mathcal{H}) \geq 4$, I.E., THAT A SET OF FOUR ELEMENTS FROM $\mathbb{R}$ CAN BE SHATTERED



Remember that we had to try to do all the possible classifications

UPPER BOUND



NO UNION OF TWO INTERVALS CAN CAPTURE THE SUBSET $\{x_1, x_3, x_5\}$ BECAUSE CAPTURING IT WOULD REQUIRE AT LEAST THREE INTERVALS.

AS A CONSEQUENCE, $VC(\mathcal{H}) = 4$.

Exercise 2

Exercise 2 asks it is to prove a fundamental property of the **VC-dimension**: monotonicity. Specifically, it asks you to prove that if one representation class is a subset of another, its VC-dimension cannot be

larger than the other's.

Exercise 2: Problem Statement

Prove that if $H \subseteq H'$, then $VC(H) \leq VC(H')$.

In plain terms, this means that if a set of hypotheses (H) is contained within a larger set of hypotheses (H'), the larger set is at least as "expressive" as the smaller one in terms of the maximum number of points it can shatter.

Explanation of the Solution

The proof relies directly on the mathematical definition of VC-dimension:

$$VC(H) = \max_{S \subseteq X} \{ |S| \mid S \text{ is shattered by } H \}$$

Here is the step-by-step logic of the proof as presented:

- 1. Define VC-dimension for H :** By definition, $VC(H)$ is the size of the largest set S that can be shattered by the hypothesis class H .
- 2. Apply the Subset Relationship:** Note that we can use the definition to compare the two classes. If a set S is shattered by H , it means that for every possible labeling of the points in S , there is a hypothesis in H that matches that labeling.
- 3. Inheritance of Shattering:** Because $H \subseteq H'$, any hypothesis available in H is also available in H' . Therefore, if H is capable of shattering a set S , H' must also be capable of shattering that same set S because it contains all the same functions (and potentially more).
- 4. Compare the Maximums:**
 - The set of all S shattered by H is a subset of the set of all S shattered by H' .
 - Consequently, the maximum size of a set shattered by H cannot exceed the maximum size of a set shattered by H' .

Mathematical Summary:

$$\begin{aligned} VC(H) &= \max_{S \subseteq X} \{ |S| \mid S \text{ is shattered by } H \} \\ &\leq \max_{S \subseteq X} \{ |S| \mid S \text{ is shattered by } H' \} \\ &= VC(H') \end{aligned}$$

Exercise 3 (I think this is useless)

Exercise 3 focuses on the **VC-dimension of the union of two representation classes** (H and H'). The exercise presents a double inequality that bounds the VC-dimension of this union:

$$VC(H), VC(H') \leq VC(H \cup H') \leq VC(H) + VC(H') + 1$$

1. The Left-Hand Side (LHS)

The left-hand side ($VC(H), VC(H') \leq VC(H \cup H')$) is "easy" because it is a direct consequence of the proof in **Exercise 2**.

Exercise 2 proves that if one hypothesis class is a subset of another ($H \subseteq H'$), then its VC-dimension must be less than or equal to the larger class ($VC(H) \leq VC(H')$). Since both H and H' are individual subsets of their union ($H \cup H'$), the VC-dimension of the union must be at least as large as the VC-dimension of either original class.

2. The Right-Hand Side (RHS)

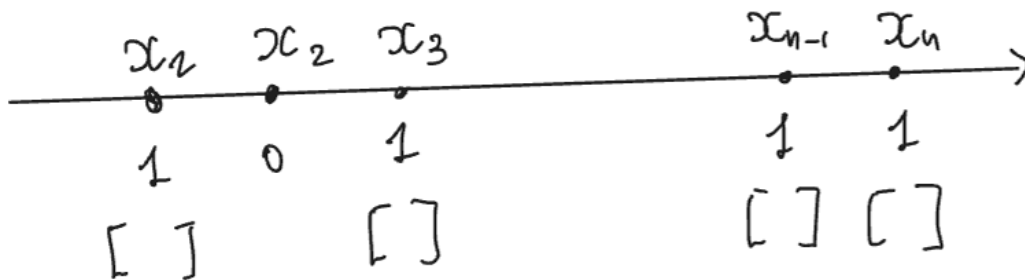
The right-hand side ($VC(H \cup H') \leq VC(H) + VC(H') + 1$) establishes an **upper bound**. It states that the VC-dimension of the combined class cannot exceed the sum of the individual VC-dimensions plus one. While the professor provides the theorem, it mentions that proving this specific upper bound is "**more complicated**" than proving the lower bound.

Exercise 4

In Exercise 4, the goal is to evaluate the **VC-dimension** for the representation class consisting of **finite unions of intervals** on the real line.

The exercise demonstrates that the VC-dimension for this class is **infinite** based on the following logic:

- **Shattering any finite set:** To prove the VC-dimension is infinite, you must show that for any integer n , there exists a set S of size n that can be **shattered**. A set is shattered if every possible subset of those points (every possible labeling of 0s and 1s) can be represented by the hypothesis class—in this case, a finite union of intervals.
- **The Construction:** Consider any finite subset S of real numbers, ordered such that $x_1 < x_2 < \dots < x_n$.
- **Capturing Subsets:** For any arbitrary subset $S' \subseteq S$ (any combination of points you want to label as "1"), you can simply place a small, individual interval around each point x_i that belongs to S' .



- **Result:** Because each point in S' is surrounded by its own interval and the points are distinct, these intervals will not capture any points that are supposed to be labeled "0".
- **Conclusion:** Since a finite union of intervals can capture **any** subset of points from **any** finite set S , no matter how large n is, the VC-dimension is not bounded by any finite number. Therefore, $VC(H) = \infty$.

Exercise 5

The Goal: Calculating $VC(H)$ for Triangles

In this exercise, the hypothesis class H consists of all possible triangles in a two-dimensional plane (R^2). The goal is to find the maximum number of points that can be "shattered" by this class.

As discussed, a triangle acts as a **geometric classifier**. For any set of points in the plane, a triangle defines a subset by "capturing" the points that fall inside its boundaries (assigning them a value of 1) and excluding those that fall outside (assigning them a value of 0).

The Result

the VC-dimension for the class of triangles is **7**.

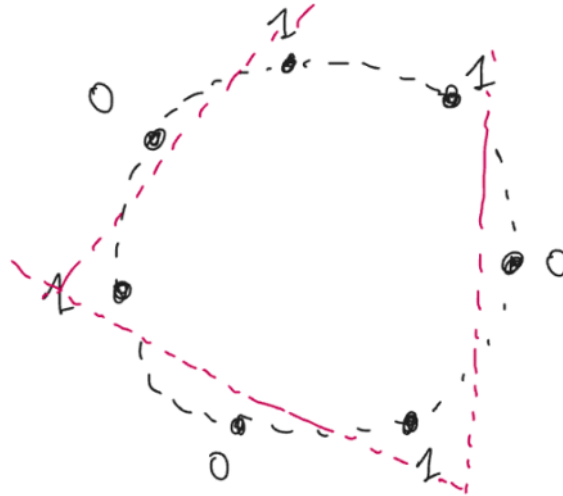
To prove that the VC-dimension is 7, we must satisfy two conditions:

1. **Lower Bound:** Show there exists at least one set of 7 points that can be shattered.
2. **Upper Bound:** Show that no set of 8 points can be shattered.

1. The Lower Bound: Shattering 7 Points

To prove the VC-dimension is at least 7, the professor provides a specific construction to show that a set of seven points can be shattered. This means that for these specific seven points, we can draw a triangle to capture **any** of the 2^7 (128) possible subsets.

- **The Arrangement:** You must place the seven points in R^2 **around a circle**.
- **The Spacing:** These points should be placed at a **constant radial distance** from one another, essentially forming the vertices of a regular heptagon.
- **The Mechanism:** Because the points are arranged on a convex curve (a circle), you have maximum flexibility. By adjusting the size, rotation, and position of a triangle, you can "reach in" and grab any combination of those points. For example, the diagram shows a dashed triangle capturing a specific group of points while leaving others outside its perimeter.



2. The Upper Bound: Why Not 8?

The professor notes that proving the upper bound—that **no set of 8 points** can be shattered by a triangle—is "**highly nontrivial**".

In computational learning theory, proving an upper bound is often more difficult because you must prove a negative: that regardless of how you arrange 8 points in a 2D plane, there will always be at least one subset of those points that a triangle simply cannot capture without accidentally including a point you wanted to exclude or excluding a point you wanted to include.

Summary of Exercise 5

- **Instance Space (X):** The 2D plane (R^2).
- **Hypothesis Class (H):** All possible triangles.
- **VC-dimension:** 7.
- **Key Takeaway:** By arranging points in a circle, we can prove we can shatter 7 points, but the geometry of a 3-sided polygon (a triangle) is mathematically limited and cannot shatter any configuration of 8 points.

Exercise 6

The classifier of this exercise, takes an input string x from the domain $X = \{0, 1\}^d$, which is the set of all possible **binary strings of length d** . For any given input string, the classifier must decide whether it

belongs to a "1" category or a "0" category.

The classification rule is determined by a fixed "parameter" string a . The classifier, $h_a(x)$, calculates the **inner product** between your input x and the parameter string a . In this context, the inner product is the **XOR sum** of the bits where both strings have a '1'.

- **If the result is 1:** The string is classified as a "1" (the parity of matching bits is odd).
- **If the result is 0:** The string is classified as a "0" (the parity of matching bits is even).

Essentially, this classifier is trying to determine if a specific subset of bits in your input (the bits "masked" by a) contains an odd or even number of 1s.

The Professor's Mathematical Example

The notes provide a specific example of calculating the inner product (the classification result) for two strings of length $d = 4$:

- **Parameter String (a):** 0110
- **Input String (x):** 0010

The classifier $h_a(x)$ performs a bitwise multiplication and then XORs the results together:

1. **Bit 1:** $0 \times 0 = 0$
2. **Bit 2:** $1 \times 0 = 0$
3. **Bit 3:** $1 \times 1 = 1$
4. **Bit 4:** $0 \times 0 = 0$

Now, sum these results using XOR (parity): $0 \oplus 0 \oplus 1 \oplus 0 = 1$. Because the result is **1**, this input string is classified as "1" (or "Positive") by this specific classifier.

A Conceptual Way to Think About It

You can think of the string a as a "**Selection Mask**."

- The 1s in string a tell the classifier which positions in the input string x it should "look at".
- In the professor's example ($a = 0110$), the 1s are at the **2nd and 3rd positions**. This means the classifier ignores everything except those two spots.
- The classifier then counts how many 1s are in those specific spots in the input x .
- If the count of 1s in those spots is **odd**, the result is **1**.
- If the count of 1s in those spots is **even**, the result is **0**.

Resolution

To determine the **lower bound** for the VC-dimension of parity functions, the goal is to prove that there exists at least one set of d points that can be **shattered**. In this context, shattering means that for any arbitrary subset of these d points you choose, there is a parity function h_a that will classify exactly those points as "1" and all others as "0".

Here is the step-by-step process of how that lower bound is established:

LOWER BOUND

1. Choosing the Set (S)

The professor uses a set S consisting of the **unit vectors** in d -dimensional space. These are binary strings of length d where only one bit is a "1" and all others are "0":

- $e_1 = (1, 0, 0, \dots, 0)$
- $e_2 = (0, 1, 0, \dots, 0)$
- ... and so on, up to $e_d = (0, 0, 0, \dots, 1)$.

This gives us a set S with exactly **d points**.

2. Defining a Target Subset (S')

To prove the set is shattered, we must be able to pick **any** subset of these points (let's call it S') and find

a classifier h_a that "captures" it. For example, if we want e_1 and e_3 to be labeled "1" and everything else to be "0", our target subset S' would be e_1, e_3 .

3. Constructing the Classifier (a)

To capture our target subset S' , we construct a specific parameter string a using a simple rule:

- The j -th bit of a is set to 1 if the unit vector e_j is in our target subset S' .
- The j -th bit of a is set to 0 if e_j is NOT in our target subset.

4. Verifying the Result

The final step is checking if this classifier h_a actually works for an input e_j . Recall that the classification function is $h_a(e_j) = a \odot e_j$ (the XOR sum of the bits).

Because e_j is a unit vector (it only has a 1 at position j), the inner product $a \odot e_j$ will **always be equal to the j -th bit of a** .

- If we put e_j in our target subset, we set the j -th bit of a to 1, so $h_a(e_j) = 1$.
- If we left e_j out, we set the j -th bit of a to 0, so $h_a(e_j) = 0$.

Conclusion: Since we can construct a string a to produce any possible combination of 0s and 1s for these d unit vectors, the set is shattered. This proves that the **VC-dimension is at least d** .

UPPER BOUND

To establish the **upper bound** for Exercise 6, the professor explains why it is impossible to shatter any set of points that is "bigger" than d . Specifically, it proves that if you have a set S containing **$d + 1$ strings**, that set cannot be shattered by the class of parity functions.

The reasoning follows a logical path using concepts from **linear algebra**:

1. The Principle of Linear Dependence

A fundamental rule of linear algebra is that in a space of dimension d , any collection of $d + 1$ vectors must be **linearly dependent**.

- This means that if you have $d + 1$ strings, at least one of those strings can be written as a "linear combination" (a XOR sum) of the others.
- For example, if you have vectors $x_1, x_2, \dots, x_{d+1}$, there exists a relationship where one vector, say x_{d+1} , is actually the result of XORing some subset of the other vectors together.

2. Parity Functions are Linear

The professor notes that **parity functions are linear**. This is a crucial property. In linear algebra, a linear function h has a specific behavior: the value of the function on a sum of vectors is equal to the sum of the values of the function on those individual vectors.

- Mathematically: $h(x \oplus y) = h(x) \oplus h(y)$.

3. Why Shattering Fails

Because parity functions are linear, the classification (the 0 or 1 label) of that $(d + 1)$ -th vector is **strictly determined** by the labels of the other d vectors.

- If x_{d+1} is a combination of x_1 and x_2 , then $h_a(x_{d+1})$ **must** be equal to $h_a(x_1) \oplus h_a(x_2)$.
- You no longer have the freedom to assign an arbitrary label to x_{d+1} once you have chosen the labels for x_1 and x_2 .

Conclusion of the Upper Bound

Shattering requires the ability to produce **every possible combination** of labels for the set. Because the label of the $(d + 1)$ -th point is "forced" by the linear relationship of the vectors, there will always be at least one configuration of 0s and 1s (a subset of S) that no parity function can ever capture.

Therefore, a set of size $d + 1$ cannot be shattered, proving that the **VC-dimension cannot be greater than d** . Combined with the lower bound proof, this confirms that the VC-dimension is exactly d .